

Самостійна робота

Тема: «Методологія розроблення програмного продукту з використанням технологій ООП. Рефакторинг».

Виконав: Устич Максим

Група : alk-43

Київ 2026р.

Відповіді на контрольні завдання та запитання

1. Що таке рефакторинг?

Рефакторинг — це процес покращення внутрішньої структури програмного коду без зміни його зовнішньої поведінки. Тобто програма після рефакторингу повинна працювати так само, як і раніше, але сам код стає більш зрозумілим, зручним для підтримки та подальшого розвитку.

Наприклад, якщо у програмі є дуже довгий метод, його можна розділити на декілька менших методів. Функціональність при цьому не зміниться, але код стане простішим для читання.

Приклад до рефакторингу:

```
void printUser()  
{  
    cout << "Ім'я: Максим" << endl;  
    cout << "Група: alk-43" << endl;  
    cout << "Спеціальність: Інженерія програмного  
забезпечення" << endl;  
}
```

Після рефакторингу:

```
void printName()  
{  
    cout << "Ім'я: Максим" << endl;  
}  
  
void printGroup()  
{  
    cout << "Група: alk-43" << endl;  
}
```

```
}
```

```
void printSpeciality()  
{  
    cout << "Спеціальність: Інженерія програмного  
забезпечення" << endl;  
}
```

У другому варіанті код краще структурований.

2. Які ознаки поганого коду?

Ознаки поганого коду часто називають “code smells”, тобто “запахи коду”. Це не обов’язково помилки, але це ознаки того, що код може бути складним для підтримки.

До основних ознак поганого коду можна віднести:

1. **Дублювання коду** — однакові фрагменти повторюються у різних місцях програми.
2. **Занадто довгі методи** — метод виконує багато різних дій і його важко читати.
3. **Занадто великий клас** — клас має багато методів і відповідає одразу за декілька задач.
4. **Складні умови** — велика кількість вкладених `if`, `else`, `switch`.
5. **Незрозумілі назви змінних і методів** — наприклад, `a`, `b`, `x1`, якщо з контексту не зрозуміло їх призначення.
6. **Надмірна залежність між класами** — зміна одного класу змушує змінювати багато інших.

Приклад поганого коду:

```
int a = 500;  
int b = 20;  
cout << a - b;
```

Краще:

```
int orderPrice = 500;  
int discount = 20;  
cout << orderPrice - discount;
```

У другому прикладі зрозуміло, що саме означають змінні.

3. Які ви знаєте методи рефакторингу?

Існує багато методів рефакторингу. Найпоширеніші з них:

1. Перейменування змінних, методів і класів

Використовується, коли назва не пояснює призначення елемента.

```
int x = 100;
```

Краще:

```
int userAge = 100;
```

2. Виділення методу

Якщо частина коду виконує окрему дію, її можна винести в окремий метод.

```
void showInfo()  
{  
    cout << "Ім'я: Максим" << endl;
```

```
        cout << "Група: alk-43" << endl;
    }
```

Можна розділити:

```
void showName()
{
    cout << "Ім'я: Максим" << endl;
}
```

```
void showGroup()
{
    cout << "Група: alk-43" << endl;
}
```

3. Виділення класу

Якщо один клас виконує забагато задач, частину функціональності можна перенести в інший клас.

4. Заміна умовних операторів поліморфізмом

Якщо є великий `switch`, який перевіряє тип об'єкта, його можна замінити ієрархією класів.

5. Усунення дублювання коду

Однакові фрагменти коду переносяться в окремий метод або клас.

4. Коли треба замінювати оператор `switch` поліморфізмом?

Оператор `switch` варто замінювати поліморфізмом тоді, коли він часто перевіряє тип об'єкта або варіант поведінки, а при додаванні нового типу доводиться постійно змінювати старий код.

Приклад до рефакторингу:

```
void drawShape(int type)
{
    switch (type)
    {
        case 1:
            cout << "Малюємо коло" << endl;
            break;
        case 2:
            cout << "Малюємо квадрат" << endl;
            break;
    }
}
```

Якщо додати нову фігуру, треба змінювати метод `drawShape()`.

Краще використати поліморфізм:

```
class Shape
{
public:
    virtual void draw() = 0;
};

class Circle : public Shape
{
public:
    void draw()
    {
        cout << "Малюємо коло" << endl;
    }
}
```

```

    }
};

class Square : public Shape
{
public:
    void draw()
    {
        cout << "Малюємо квадрат" << endl;
    }
};

```

Тепер для додавання нової фігури не потрібно змінювати старий код. Достатньо створити новий клас.

5. Яким чином та як часто треба виконувати рефакторинг?

Рефакторинг треба виконувати поступово і регулярно. Його не варто відкладати до кінця розробки, тому що з часом код стає складнішим, і виправляти його буде важче.

Рефакторинг доцільно виконувати:

- після написання нового функціоналу;
- перед додаванням складних змін;
- після виявлення дублювання коду;
- коли метод або клас стає занадто великим;
- коли код важко читати або тестувати.

Головне правило: після рефакторингу поведінка програми не повинна змінитися. Тому бажано перевіряти програму після кожної зміни.

Приклад простого рефакторингу:

Було:

```
cout << price - discount;
```

Стало:

```
int finalPrice = price - discount;  
cout << finalPrice;
```

Код виконує те саме, але став зрозумілішим.

Приклад рефакторингу у Visual Studio

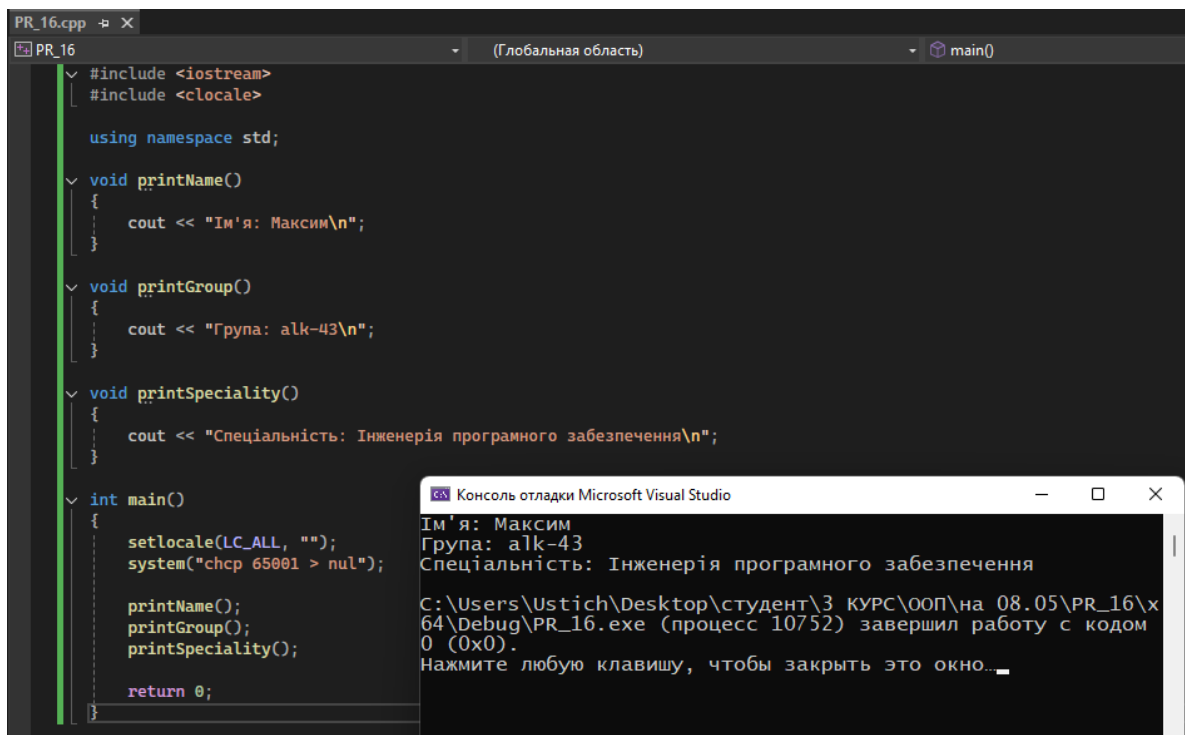


Рис. 1 – Приклад рефакторингу з виділенням окремих методів у Visual Studio